Response to NIST NCCoE Concept Paper

Accelerating the Adoption of Software and AI Agent Identity and Authorization

Submitted to: AI-Identity@nist.gov

Date: February 21, 2026

Submitted by: Santosh T, Creator of AstraCipher

LinkedIn: https://www.linkedin.com/in/santechie21

Project Website: https://astracipher.com

Open Source: https://github.com/san-techie21/astracipher

Executive Summary

We appreciate the opportunity to respond to the NCCoE concept paper "Accelerating the Adoption of Software and AI Agent Identity and Authorization." This response draws from our experience building **AstraCipher**, an open-source protocol and SDK that provides cryptographic identity for AI agents using W3C Decentralized Identifiers (DIDs), W3C Verifiable Credentials, and NIST-standardized post-quantum cryptography (FIPS 203 / FIPS 204).

We offer this response in three parts:

1. **Use cases and real-world challenges** we have observed in the field
2. **Technical recommendations** addressing the specific questions raised in the concept paper
3. **An open-source reference implementation** available for NCCoE lab demonstration

Our central thesis: AI agents require a fundamentally different identity model than human users or traditional software services. Bearer tokens, OAuth sessions, and workload identities were designed for human-initiated, session-based interactions. Autonomous agents that plan, delegate, and operate across organizational boundaries require self-sovereign, capability-bounded, cryptographically verifiable identities — with post-quantum resistance built in from the foundation.

---

1. Use Cases and Opportunities

1.1 Multi-Agent Financial Trading

AI agents in financial services autonomously execute trades, manage portfolios, and interact with multiple broker APIs. A trading agent spawns sub-agents for market analysis, order execution, and risk assessment. Each sub-agent calls external APIs on behalf of the parent. Current approaches issue a single API key or OAuth token to the parent agent, which is then shared with sub-agents — creating a flat, unauditable trust model.

What is needed: Each agent and sub-agent requires its own verifiable identity with a cryptographic chain of delegation back to the human portfolio manager. Credentials must encode capability boundaries (e.g., "can read

market data for equity/RELIANCE but cannot execute trades exceeding $10,000").

1.2 Healthcare Agent Coordination

AI agents in healthcare coordinate across hospital systems, insurance providers, and pharmacy networks. A patient-facing triage agent needs to request lab results from Hospital A, check insurance coverage with Provider B, and schedule a follow-up with Clinic C. Each system needs to verify the agent's identity, the patient's consent delegation, and the agent's authorized scope — without calling a central identity server that may introduce latency or become a single point of failure.

What is needed: Offline-verifiable credentials containing the agent's identity, delegated consent proof, and fine-grained capability boundaries — verifiable using only the issuer's public key.

1.3 MCP and A2A Protocol Ecosystems

The Model Context Protocol (MCP) and Google's Agent2Agent (A2A) protocol are rapidly becoming the standard interfaces for agent communication. Both currently rely on OAuth 2.1 / OpenID Connect for authorization. OAuth answers "is this session authorized?" but not "who is this agent, what are its capabilities, and who is accountable for its actions?"

What is needed: A credential-based identity layer that complements OAuth — where agents carry self-describing, cryptographically signed credentials that can be verified offline by any peer.

2. Responses to Concept Paper Questions

2.1 How should agents be recognized in enterprise architectures?

Recommendation: Treat agents as first-class identity principals with W3C Decentralized Identifiers (DIDs).

AI agents should not be modeled as extensions of human user accounts or as service accounts. They should receive globally unique, self-sovereign identifiers following the W3C DID specification:

- **DID Method:** A purpose-built DID method (e.g., `did:astracipher:mainnet:<fingerprint>`) that encodes the agent's cryptographic public keys directly in the identifier, enabling resolution without a centralized registry.
- **DID Documents:** Each agent's DID Document should contain its verification methods (public keys), supported cryptographic algorithms, and service endpoints.
- **Network Segmentation:** The DID method should support network qualifiers (e.g., testnet, mainnet, enterprise) to enable organizational isolation while maintaining global uniqueness.

2.2 What metadata is essential for an AI agent's identity?

Recommendation: Verifiable Credentials that encode capabilities, permissions, trust level, and delegation chain.

An agent's identity metadata should be carried in a W3C Verifiable Credential containing:

Metadata Field	Purpose	Example
`id`	Globally unique credential ID	`urn:uuid:a3b4c5d6-...`
`issuer`	DID of credential issuer	`did:astracipher:mainnet:c2e3...`
`credentialSubject.id`	DID of the agent	`did:astracipher:mainnet:4bea...`
`credentialSubject.name`	Human-readable agent name	`portfolio-trading-agent`
`credentialSubject.capabilities`	Enumerated capabilities	`["market-data:read", "orders:execute"]`
`credentialSubject.trustLevel`	Numeric trust score (1-10)	`7`
`credentialSubject.permissions`	Fine-grained access rules	`[{"resource": "equity/*", "actions": ["read"]}]`
`issuanceDate` / `expirationDate`	Temporal validity	ISO 8601 timestamps
`proof`	Cryptographic signature(s)	Hybrid PQC + classical

This credential is **self-contained**: any verifier with access to the issuer's public key can verify its authenticity, integrity, and expiration — without contacting an external server.

2.3 Should agent identities be ephemeral or fixed?

Recommendation: Support both — short-lived credentials on long-lived identities.

- **Persistent DID (identity):** Created when the agent is provisioned. Survives across tasks and sessions. Revocable by the issuing organization.

- **Short-lived Credential (authorization):** Issued per-task or per-session with explicit expiration (hours to days). Capability-bounded to the specific task. Automatically invalid after expiry.

This separation mirrors the human model: a person has a persistent identity (passport) but receives short-lived authorization (boarding pass) for specific actions.

2.4 Should identities be tied to hardware, software, or organizational boundaries?

Recommendation: Organizational boundaries as the primary trust anchor.

- **Organization-tied:** An agent's DID should be issued by an organizational DID that serves as the root of a trust chain. This enables enterprises to revoke all agent identities if compromised, establish bilateral trust between organizations, and audit all agent actions.
- **Software attestation (optional):** Runtime, model version, and SDK version as supplementary claims.
- **Hardware binding:** Optional, reserved for high-assurance environments only.

2.5 How should delegation chains be handled?

Recommendation: Cryptographic trust chains with depth limits and capability attenuation.

When Agent A spawns Sub-Agent B, and B spawns Sub-Agent C:

Human Sponsor → Agent A → Sub-Agent B → Sub-Agent C

Each delegation is recorded as a verifiable credential where the parent's credential is referenced in the child's proof, capabilities can only be **attenuated** (narrowed, never expanded), and a configurable maximum chain depth prevents unbounded delegation.

2.6 What controls should prevent prompt injection?

Recommendation: Credential-bound capability boundaries as defense-in-depth.

Even if an agent is prompt-injected and attempts unauthorized actions, the credential's capability boundaries limit what it can do. A market-data agent with read-only equity permissions cannot execute trades regardless of injected instructions. Because every action is tied to a cryptographic identity, injected actions can be attributed and audited.

2.7 Post-quantum cryptographic readiness

Recommendation: Hybrid signatures (PQC + classical) from day one.

Any identity framework standardized in 2026 should mandate post-quantum readiness:

- **Hybrid signing:** ML-DSA-65 (FIPS 204) + ECDSA P-256. Both must verify. Provides quantum resistance, backward compatibility, and defense-in-depth.
- **Hybrid key encapsulation:** ML-KEM-768 (FIPS 203) + ECDH P-256 for encrypted agent-to-agent communication.
- **Rationale:** Agent credentials are long-lived artifacts that may be verified years from now. "Harvest now, decrypt later" attacks mean classical-only signatures are already insufficient.

3. Reference Implementation: AstraCipher Protocol

We have built and open-sourced a complete implementation of the architecture described above.

3.1 Availability

Package	Registry	Install Command
@astracipher/core	npm	`npm install @astracipher/core`
@astracipher/crypto	npm	`npm install @astracipher/crypto`
@astracipher/mcp-server	npm	`npm install @astracipher/mcp-server`
@astracipher/a2a-adapter	npm	`npm install @astracipher/a2a-adapter`
@astracipher/cli	npm	`npm install -g @astracipher/cli`
astracipher	PyPI	`pip install astracipher`

- **Live demo:** https://astracipher.com (real post-quantum cryptography in the browser)
- **Whitepaper:** https://astracipher.com/whitepaper
- **Source:** https://github.com/san-techie21/astracipher

3.2 NCCoE Lab Demonstration Offer

We offer AstraCipher as a candidate technology for the proposed NCCoE demonstration project. The SDK is open source, standards-based (W3C DID Core, W3C VC, FIPS 203/204), interoperable with MCP and A2A protocols, and ready for integration testing with OAuth 2.1, SPIFFE/SPIRE, SCIM, and NGAC.

We are willing to participate in technical collaborations and contribute to the NCCoE Practice Guide.

4. Alignment with Referenced Standards

NIST Reference	AstraCipher Alignment
SP 800-207 (Zero Trust)	Credential verification at every agent boundary; no implicit trust
SP 800-63-4 (Digital Identity)	W3C DIDs as non-human identity principals
FIPS 204 (ML-DSA)	ML-DSA-65 hybrid signatures on all credentials
FIPS 203 (ML-KEM)	ML-KEM-768 hybrid key encapsulation
OAuth 2.1 / OpenID Connect	Complementary — credential identity on top of OAuth sessions
SPIFFE/SPIRE	DIDs interoperate with SPIFFE IDs at workload boundary

5. Summary of Recommendations

1. Treat AI agents as **first-class identity principals** — not extensions of human accounts
2. Adopt **W3C DIDs** as the standard identifier format for agents
3. Use **W3C Verifiable Credentials** for capability-bounded authorization metadata
4. **Separate persistent identity (DID) from short-lived authorization (credential)**
5. Mandate **hybrid post-quantum signatures** (FIPS 204 ML-DSA + ECDSA)

6. Standardize **cryptographic delegation chains** with capability attenuation
7. Leverage **credential capability boundaries** as defense-in-depth against prompt injection
8. Ensure **interoperability with MCP and A2A** protocols

Santosh T

Creator of AstraCipher

LinkedIn: https://www.linkedin.com/in/santechie21

https://astracipher.com | https://github.com/san-techie21/astracipher